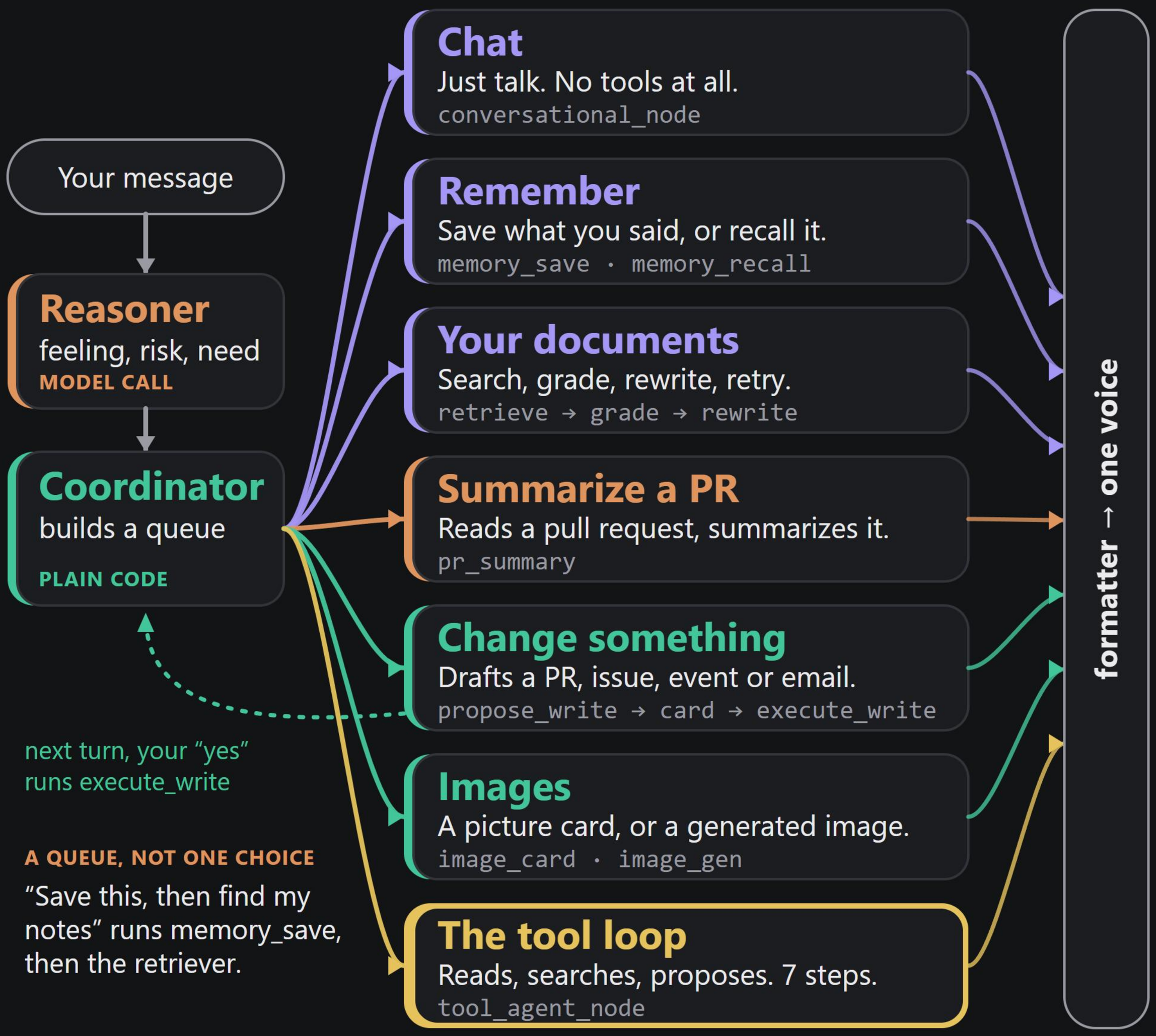
THE ROUTE

Where every message goes.

A reading step, then plain rules pick the path. The tool loop is just one branch.



Your “yes” goes first

An approval reply skips straight to the write step, ahead of every other rule.

Words decide the obvious

Commit or merge words, a picture request, a progress note: sent by code, no model call.

Flags queue the rest

The reading step’s flags add agents in order: memory, documents, summary, then the tool loop.

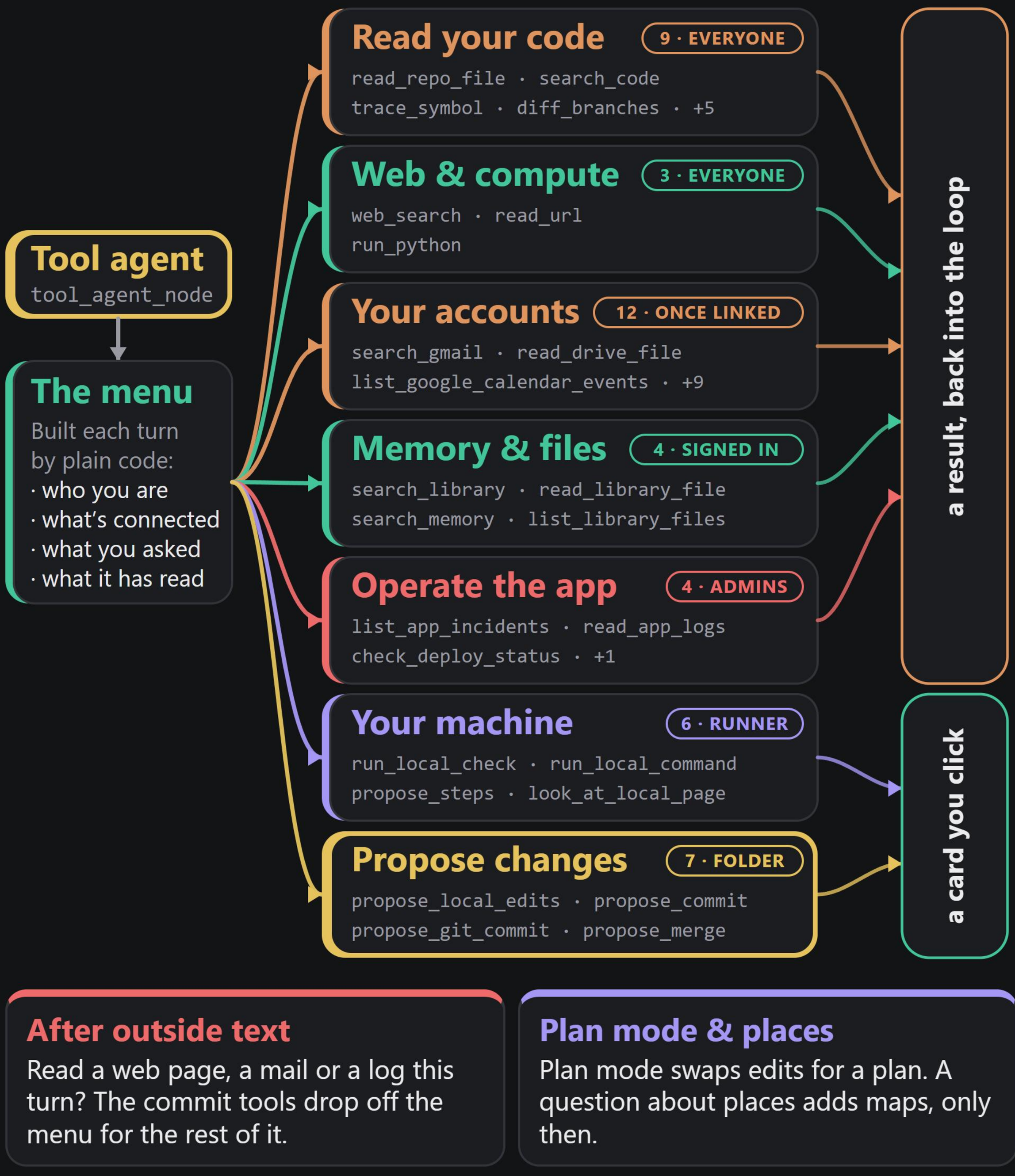
Nothing matched?

Plain chat. Every path ends at the same writer, so it always sounds like one person.

THE MENU

It never sees all 45 tools.

Each turn the code fans out to a subset. A tool that isn’t on the menu can’t be called.



After outside text

Read a web page, a mail or a log this turn? The commit tools drop off the menu for the rest of it.

Plan mode & places

Plan mode swaps edits for a plan. A question about places adds maps, only then.

THE LOOP

Pick. Read. Decide. Repeat.

Inside the tool loop the model works one step at a time and never guesses what a tool returned.

1 **It picks tools**
Several at once when they’re independent: **eight reads can cost two steps.**

▼

2 **Real results come back**
Labelled as data, never instructions. Results older than two steps shrink, so the loop stays cheap.

▼

3 **It decides**
Another step, an answer, or a proposal. Asking for the same thing a fourth time is refused.

▼

4 **Answer, or propose**
Anything that changes something is a **proposal you click**, not an action it takes.

Three kinds of tool

Look
Reads and searches. Each step spends budget.
read-only

Propose
An edit, commit or merge as a card. You click. Costs no budget.
needs your yes

Run
On your own machine, after you press Run. Your computer checks again.
approved twice

The model chooses. The code decides what’s allowed.

THE STEP BUDGET

Reading spends steps. Doing earns them back.

Every turn gets a small budget. That’s how it stays fast, cheap and unable to run away.

1 Start small

7 steps a turn, 14 in Deep mode, never more than 12 at once. A change request gets at least 6.

2 Don’t read forever

On longer budgets, at **⅓ and ⅔** it must choose: answer now, or name what it still needs. At 2 steps left it asks for everything at once.

3 Acting earns steps back

An edit that was applied, or a check that ran, hands back **+4**. The same result again earns +2, +1, then nothing. A new failing test is a counter-hit: room to read what the fix needs.

4 The chain ends when it stops landing

A step with no real action earns nothing. A hard cap of **30** steps handed back per turn.

One coding turn illustrative

7 R R E T E T ✓

R read **E** edit applied (+4) **T** test failed (counter-hit) **T** test passed ✓ answer

Edit, test, fix, test: more steps than the budget alone allows, but only while every step lands.

A second reader

After an applied edit, a cheap model checks it. A real finding hands steps back, twice at most.

Low power

Past the spend limit the budget drops to 4 steps and the extras pause.

WHY YOU CAN TRUST IT

Autonomy with brakes.

The more an assistant can do, the less it should be able to do on its own.

1 **Propose, then click**

Edits show a diff and an Apply button. Commands show the exact line and a Run button. Merges and commits are cards. **Nothing changes silently.**

2 **Two checks, same rules**

A command is checked on the server and again on your machine, against the **same rules file**. Your machine decides last. Tests and servers are asked for by name, never by command.

3 **Outside text is data**

After it reads a web page, an email, a log or a command’s output, **commits are refused that turn.** Hidden instructions can’t become actions.

4 **A browser that can’t phone out**

It looks at pages on your own computer only. Every request to anywhere else is blocked, and a test proves it with a real network address.

5 **Spend brakes**

Hard daily caps per person and overall, a low-power mode, and cheap models for the loop. A test run can’t spend real money either.

Every rule here came from a real failure.